

Online Boutique

End-to-End DevOps & Cloud Platform Engineering Project

Docker · Kubernetes · Helm · GitHub Actions · Terraform · Azure AKS · ArgoCD · Prometheus · Grafana

Project Documentation & Engineering Log

Prepared for technical interviews and portfolio review

1. Project Overview

This project takes Google's open-source Online Boutique microservices demo (11 services across multiple languages, communicating over gRPC) and rebuilds its entire deployment platform from scratch, rather than reusing the maintainers' pre-built manifests. The goal was to personally design and operate every layer of a production-style delivery pipeline: containerization, orchestration, packaging, CI/CD, infrastructure-as-code, GitOps, and observability.

Starting point: the upstream Kubernetes manifests, Helm chart, Terraform, Istio config, and GitHub Actions workflows were deliberately removed, keeping only the microservice source code and gRPC protobuf contracts. Everything downstream of that point — the container build process, the Kubernetes cluster, the Helm chart, the CI pipeline, the cloud infrastructure, the GitOps sync, and the monitoring stack — was written from first principles.

Objectives

- Understand containers and Kubernetes deeply enough to debug them under failure, not just deploy them.
- Package the application with a hand-written Helm chart instead of an inherited one.
- Build a CI pipeline that compiles and publishes all 11 service images automatically.
- Provision real cloud infrastructure (Azure AKS) using Terraform, with a clear understanding of state.
- Implement GitOps with ArgoCD so the cluster's state is driven by Git, not manual kubectl commands.
- Add metrics, dashboards, and alerting so system health is observable, not guessed at.

Project Timeline

Day	Focus	Key Deliverable
Day 1	Docker, Kubernetes fundamentals, deploy & break the app	Running storefront on a local kind cluster + failure-mode notes
Day 2	Helm chart authored from scratch, CI pipeline	Custom Helm chart + green GitHub Actions build to GHCR
Day 3	Terraform, Azure AKS, real cloud deployment	Live public IP on AKS, infra fully in code
Day 4	ArgoCD GitOps, Prometheus/Grafana, README	Self-healing GitOps deployment + monitoring dashboards

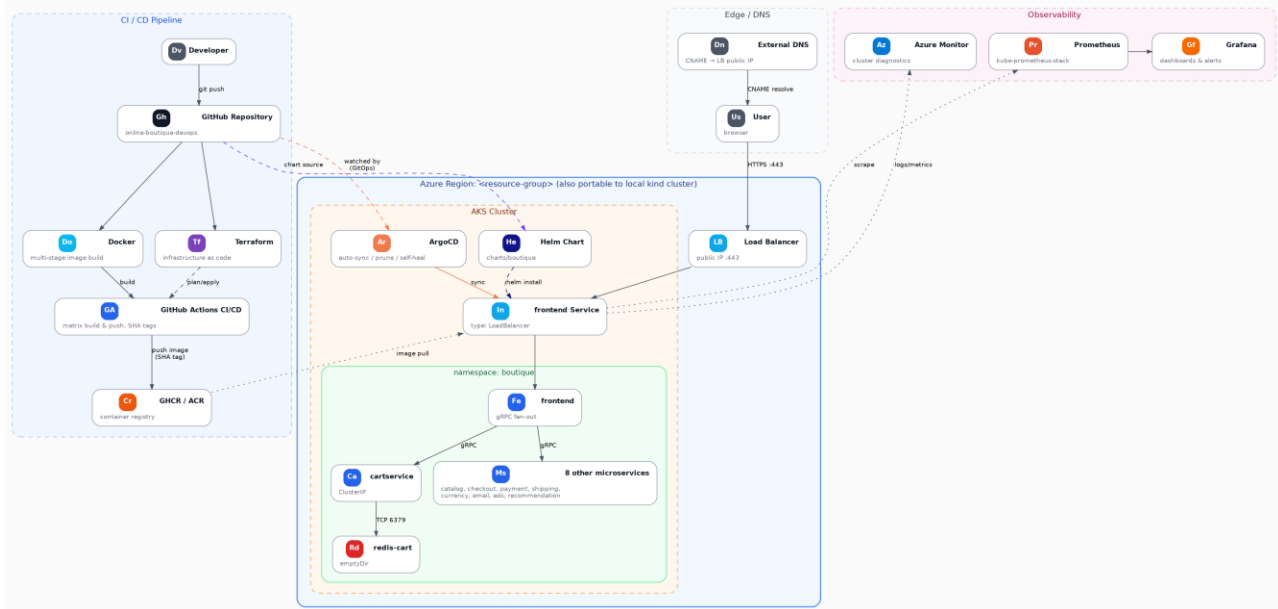
Project Evolution

Each day adds one layer on top of what already works — nothing from a previous day is thrown away, only replaced with a better version of itself (raw manifest → Helm chart; local kind → real AKS; manual kubectl → GitOps).



2. Architecture

The diagram below shows the full target platform: local development in GitHub Codespaces, a CI pipeline that builds and publishes container images, a hand-written Helm chart, infrastructure provisioned by Terraform on Azure AKS, GitOps delivery via ArgoCD, and a Prometheus/Grafana observability layer sitting on top of the 11-service application.



Request Flow

A browser talks to the frontend service over plain HTTP only. The frontend is the single entry point — it opens long-lived gRPC connections to every backend service at startup (product catalog, cart, checkout, payment, shipping, currency, email, ads, recommendation) and fans requests out over gRPC. Backend services are ClusterIP-only and unreachable from outside the cluster; only frontend is exposed as a LoadBalancer, which keeps the attack surface small and mirrors how this would be run for real users.

Two Flows to Be Able to Draw From Memory

An interviewer will usually ask for one of these two flows, unprompted, and expect you to talk while drawing on a whiteboard. They are deliberately separate: one is about how code becomes a running deployment, the other is about how a single user click gets served.

A. Deployment Flow — from git push to running pods (GitOps)

1. **Push** — A developer commits and pushes to the GitHub repository. This is the only manual step in the entire pipeline — everything after this point is automated.
2. **CI fans out** — The push triggers two independent GitHub Actions workflows in parallel: an application pipeline (Docker) and an infrastructure pipeline (Terraform). They are kept separate deliberately, so an infrastructure change can never accidentally ship untested application code, and vice versa.
3. **Build & publish** — The application pipeline runs lint/tests first, then a matrix build compiles all changed services in parallel using multi-stage Docker builds (compiler stage discarded, distroless runtime stage kept). Each image is tagged with the git commit SHA — never :latest — and pushed to GHCR/ACR.

4. **Infrastructure reconciled** — The Terraform pipeline reads remote state, computes a plan against the current Azure resources, and applies it — provisioning or updating the AKS cluster, ACR, networking, and identities. This runs independently of the application build.
5. **Desired state lives in Git** — The Helm chart (templates + values.yaml) and the ArgoCD Application manifest both live in the same Git repository as the source code. After a build, the new image tag is what changes in values.yaml — that commit is the actual deployment trigger.
6. **ArgoCD detects drift** — ArgoCD continuously watches the Git repo — it is not called by the CI pipeline directly. On each reconciliation loop it diffs the desired state (Git) against the live cluster state (what's actually running in AKS).
7. **Sync** — If a diff exists, ArgoCD applies it: installing or upgrading the Helm release, updating Deployments, Services, and ConfigMaps to match Git exactly. New pods pull their images from GHCR/ACR and roll out.
8. **Self-heal** — If someone manually edits or deletes a Kubernetes object with kubectl, the next reconciliation loop reverts it back to match Git. This is the behaviour demonstrated on Day 4 and is the single clearest way to explain 'what GitOps gives you that CI/CD alone doesn't.'

B. Runtime Flow — from browser click to response ("Add to Cart")

1. **DNS resolution** — The browser resolves the app's domain via an external DNS CNAME pointing at the Load Balancer's public IP. (On a local kind cluster this step doesn't exist — kubectl port-forward stands in for it.)
2. **Edge entry** — The HTTPS request hits the Load Balancer on :443, which forwards to the frontend Service — the only Service of type LoadBalancer in the entire application.
3. **Routed to a pod** — The frontend Service forwards to one of the running frontend pods. That pod opened long-lived gRPC client connections to every backend service by its Kubernetes DNS name (e.g. cartservice.default.svc.cluster.local) once, at startup — not per request.
4. **gRPC fan-out** — For 'Add to Cart' specifically, frontend calls AddItem on cartservice over gRPC. For checkout, frontend calls checkoutservice, which itself calls paymentservice, shippingservice, emailservice, and currencyservice.
5. **Data layer** — cartservice reads and writes the user's session cart in redis-cart over plain TCP (port 6379). Every backend Service is ClusterIP-only — resolvable and reachable only from inside the cluster, never directly from the internet.
6. **Response** — frontend assembles the result into an HTML template and returns it to the browser over plain HTTP. The browser itself never speaks gRPC — that protocol exists only between frontend and the backends.

C. Observability Flow — "how do you know it's broken?"

1. **Scrape** — Prometheus scrapes metrics endpoints exposed by pods and nodes on a fixed interval and stores them as time series (kube-prometheus-stack installs the exporters that make this work out of the box).
2. **Query & visualize** — Grafana queries Prometheus using PromQL (e.g. rate(http_requests_total[5m])) and renders the results as dashboards — pod restarts, request rate, error rate, CPU/memory per service.
3. **Alert** — Alert rules evaluate continuously against the same metrics and fire a notification when a threshold is crossed (e.g. a pod crash-looping), closing the loop from 'something is wrong' to 'someone finds out' without anyone tailing logs by hand.

60-second interview summary: "A push to Git triggers CI, which builds SHA-tagged images and publishes them to a registry, while Terraform separately reconciles the underlying Azure infrastructure. ArgoCD watches that same Git repo and continuously syncs the live AKS cluster to match it, self-healing any manual drift. At runtime, only the frontend is publicly exposed behind a LoadBalancer; it fans requests out over gRPC to ClusterIP-only backends, one of which — cartservice — persists state in Redis. Prometheus and Grafana sit on top of all of it so I can see and alert on the system's health instead of guessing."

Component-by-Component Walkthrough

Reading the diagram left to right, top to bottom:

CI/CD Pipeline (blue, left): a push to the GitHub repo fans out two ways — Docker builds and pushes every service image through GitHub Actions to GHCR/ACR, SHA-tagged; Terraform plans and applies infrastructure changes on the same trigger. The two are kept independent so an infra change can never ship untested application code.

Edge / DNS (top right): an external DNS CNAME resolves the app's domain to the Load Balancer's public IP — this is what a real deployment sits behind; on a local kind cluster this whole box is skipped in favor of kubectl port-forward.

Azure Region → AKS Cluster → namespace: boutique (nested boxes): mirrors how the app is actually isolated — a resource group containing the AKS cluster, which contains the application namespace. The same nesting exists locally as Docker (kind's node container) → kind cluster → default namespace.

Load Balancer → frontend Service: the only public entry point (HTTPS :443). Everything inside the namespace box is ClusterIP-only and reachable solely by DNS name from other pods, never directly from the internet.

frontend → cartservice / 8 other microservices: the frontend holds long-lived gRPC connections to every backend opened at startup; cartservice additionally depends on redis-cart over plain TCP for session storage.

Helm Chart + ArgoCD (inside AKS Cluster): the Helm chart is the packaging format; ArgoCD is what actually applies it — it watches the Git repo and continuously syncs the live cluster to match, which is what makes 'delete a Deployment and watch it come back' possible on Day 4.

Observability (pink, top right): Prometheus scrapes metrics from the cluster, Grafana turns them into dashboards and alerts, and Azure Monitor covers platform-level diagnostics — together this answers 'how do you know it's broken?' without reading logs by hand.

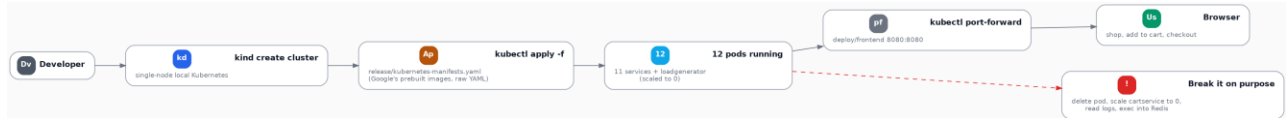
Why each layer exists

Layer	Purpose
Docker	Reproducible, minimal runtime images (multi-stage builds, distroless final stage)
kind	Fast, disposable local Kubernetes cluster for daily development
Helm	Templated, versioned packaging — one chart, many environments
GitHub Actions	Automatic build + publish on every push, SHA-tagged images
Terraform	Declarative, reproducible cloud infrastructure with reviewable state
Azure AKS	Managed Kubernetes control plane for a real production-like cluster
ArgoCD	Git as the single source of truth; automated sync + self-heal + drift correction
Prometheus/Grafana	Metrics, dashboards, and alerting — answers 'how do you know it's broken?'

Architecture Snapshot: Day 1 vs. Day 2 vs. Day 3

The full-stack diagram above is the end state. Day by day, only a slice of it exists. Seeing each slice on its own is what actually gets built and run each day, and it's a cleaner way to explain progress to someone who wasn't watching.

Day 1 — raw manifest applied directly to a local kind cluster. No packaging, no CI, no cloud — just kubectl, Kubernetes objects, and deliberate failures to learn from.



Day 2 — the raw manifest is gone, replaced by a hand-written Helm chart. A CI pipeline now builds and publishes every image with an immutable SHA tag, and the chart is repointed at those images once a build is green. Still the same local kind cluster underneath — only the packaging and the image supply changed.

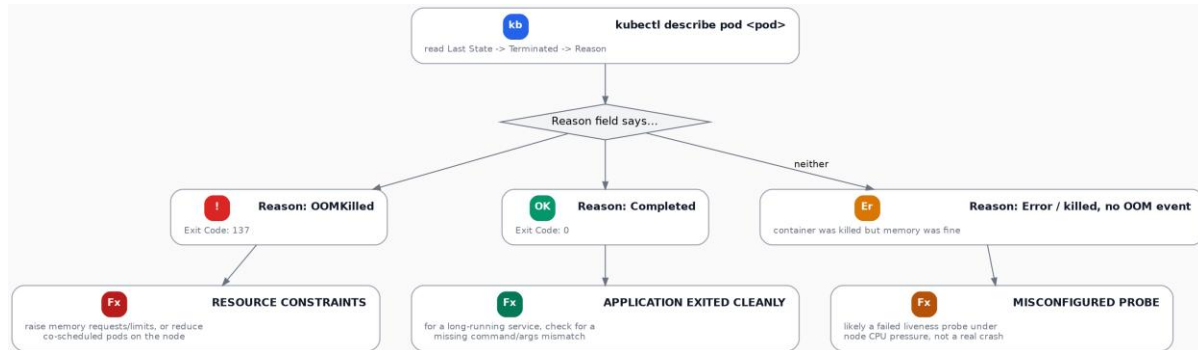


Day 3 — kind is retired for the day. The same Helm chart deploys unchanged onto a real Terraform-provisioned AKS cluster — proof the packaging was genuinely portable. This is the only day that costs money and the only day with a hard teardown step at the end of every session.

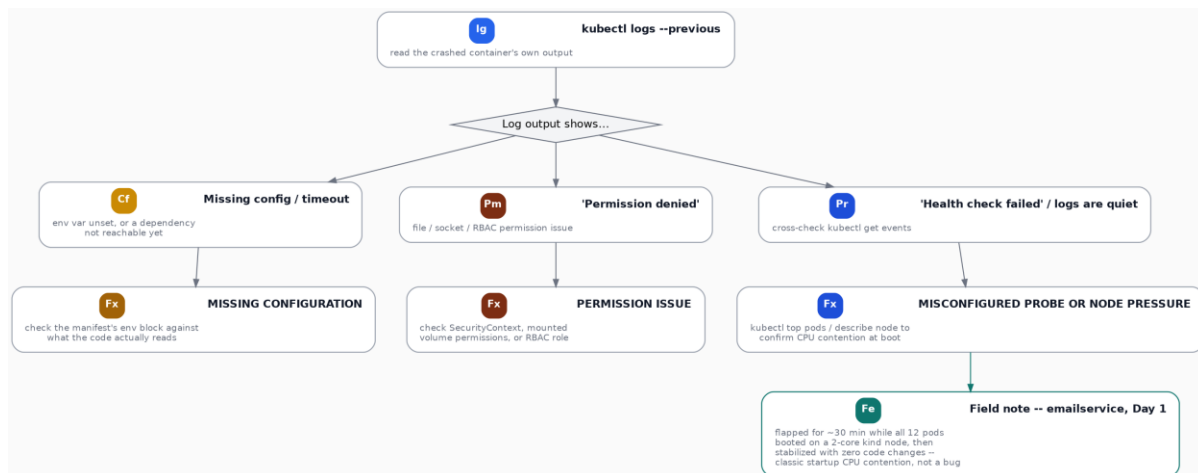


3. Debugging Playbook: Diagnosing CrashLoopBackOff

A pod flapping through CrashLoopBackOff and then quietly stabilizing on its own — as happened with emailservice on Day 1 — is common enough to be worth a repeatable diagnostic routine rather than a one-off guess. The two decision trees below are the routine: start from `kubectl describe pod`, and only reach for `kubectl logs --previous` if the Reason field alone doesn't explain it.



If the Reason field doesn't clearly explain the crash — most often because the container was killed rather than crashing on its own — the next step is to read what the container itself printed right before it died, and cross-check that against cluster-level events.



How to explain this in an interview

"CrashLoopBackOff isn't a diagnosis by itself — it just means kubelet is backing off between restart attempts, with the delay growing exponentially up to a 5-minute cap. The actual cause is in describe pod's Reason field: OOMKilled means the resource limit was too low; Completed with exit code 0 usually means the entrypoint exited early; and a kill with no OOM event usually points at a liveness probe that timed out because the node was under CPU pressure, not because the app was broken. If the Reason alone isn't conclusive, logs --previous and get events fill in the rest. In my case, emailservice flapped for about 30 minutes while all 12 pods were starting on a 2-core kind node, then stabilized on its own with zero changes from me — classic startup resource contention, not an application bug."

4. Problems Faced & How I Solved Them

This is the section interviewers care about most — it proves hands-on debugging, not just following commands. The entries below are logged as the work happens, day by day. Day 1 entries are filled in from the failure modes

deliberately triggered during that day's session; Day 2–4 entries are structured the same way so they can be completed as each phase is finished, ideally with the exact error text, command output, and screenshot pasted in.

Day 1 — Containers & Kubernetes Fundamentals

Issue 1 — Frontend container crashes immediately when run standalone

Problem: Running the frontend image directly with ``docker run`` caused it to panic and exit instead of starting a web server.

Diagnosis: Read the panic message and traced it to ``mustMapEnv()`` in `main.go` — it calls ``os.Getenv()`` for each backend address and panics with the exact variable name (e.g. ``PRODUCT_CATALOG_SERVICE_ADDR``) if it is unset.

Solution: Confirmed this was intentional fail-fast behaviour, not a bug — the container is designed to only run inside Kubernetes, where the Deployment manifest injects every ``*_SERVICE_ADDR`` environment variable. No code change needed; the fix is architectural (run it in the cluster, not standalone).

What I learned: A service refusing to start with missing config is safer than starting silently broken — it turns a 2am mystery into an immediate, readable error.

Issue 2 — cartservice outage — diagnosing a partial application failure

Problem: After scaling ``cartservice`` to zero replicas, clicking 'Add to Cart' in the browser failed, while the rest of the site (home page, product catalog) kept working.

Diagnosis: Ran ``kubectl get pods -l app=cartservice`` (zero pods) and ``kubectl logs -f deploy/frontend``, which showed a gRPC 'connection refused' error to ``cartservice:7070``. Confirmed with ``kubectl get svc cartservice`` that the Service object still existed even with no backing pods.

Solution: Understood that ``cartservice:7070`` resolves through CoreDNS to a stable ClusterIP Service, which is separate from whether any pod is actually behind it — so DNS still answers even when the service is fully down. Scaled ``cartservice`` back to 1 replica and confirmed checkout worked again.

What I learned: Kubernetes Services decouple 'can I resolve this name' from 'is anything actually there to answer.' A health check on frontend alone is not enough — you need per-dependency visibility.

Issue 3 — Pod deleted manually — unexpected identity change

Problem: Deleting a frontend pod by name and re-checking ``kubectl get pods`` showed a pod with a completely different name running in its place.

Diagnosis: Compared pod names before and after, and reviewed the Deployment/ReplicaSet relationship — the Deployment continuously reconciles the actual pod count against the desired replica count.

Solution: No action needed — this was Kubernetes working as designed. Re-ran ``kubectl port-forward`` since the forwarding session had been bound to the now-deleted pod.

What I learned: Pods are disposable and identity-less; the Deployment is the durable object. This is different from ArgoCD's self-healing (Day 4), which restores the whole Deployment spec from Git, not just pod count.

Issue 4 — Cart data lost after a Redis restart

Problem: Cart contents disappeared whenever the ``redis-cart`` pod restarted.

Diagnosis: Shelled into the pod with ``kubectl exec -it deploy/redis-cart -- redis-cli`` and ran ``KEYS *`` before and after a restart — the key space was empty afterwards. Traced this to Redis using an ``emptyDir`` volume, which is tied to the pod's lifecycle, not persistent storage.

Solution: Documented this as a known limitation rather than fixing it on Day 1 — flagged for a real deployment, Redis would need a ``PersistentVolumeClaim`` or a managed cache service.

What I learned: Distinguishing 'acceptable for a local dev cluster' from 'acceptable for production' — and being able to name the fix — reads better in an interview than pretending everything was production-ready from day one.

Issue 5 — emailservice — CrashLoopBackOff at cluster boot, self-resolved after ~30 minutes

Problem: emailservice is unused in my testing (I never trigger checkout emails), yet it showed CrashLoopBackOff shortly after kubectl apply, with no action taken on my part.

Diagnosis: Applied the debugging playbook above: kubectl describe pod first to check the Reason field, then kubectl logs --previous and kubectl top pods to check for resource pressure at the time, since all 12 pods were starting simultaneously on a 2-core kind node.

Solution: No fix was needed — the pod stabilized on its own once the initial CPU spike from all pods starting together subsided. This matched the 'misconfigured probe / node pressure' branch of the playbook rather than an actual application bug.

What I learned: CrashLoopBackOff means kubelet is backing off between restarts, not that something is permanently broken — the exponential backoff (up to a 5-minute cap) can make a transient resource issue look like a sustained failure. Kubernetes deploys every Deployment in a manifest regardless of whether a service is actually exercised by user traffic, so 'I don't use this service' doesn't mean it's exempt from health checks.

Day 2 — Helm & CI/CD

Day 2 replaces the raw manifest with a hand-written Helm chart and a CI pipeline that builds and publishes every image. Three concepts anchor the whole day, and they're worth being able to state cold:

Term	Meaning	Today
Chart	A folder of templates using {{ .Values... }}	charts/boutique
Values	The knobs — image tag, replicas, ports, env	values.yaml
Release	One installed instance, with revision history you can roll back	helm install boutique ...

Concept: Health Probes Must Match the Protocol Underneath

Every service in the chart gets a readiness and liveness probe, but not the same kind — the probe has to speak whatever protocol the service actually speaks, or Kubernetes is just sending noise it can't interpret and getting a false 'unhealthy' back.

Service	Protocol	Probe Type	What It Actually Checks
frontend	HTTP	httpGet on /_healthz	Can the web server respond to a real request?
redis-cart	Raw TCP	tcpSocket on 6379	Is the port open and accepting connections?
cartservice	gRPC	grpc on 7070	Does the gRPC health service reply SERVING?

An HTTP probe against Redis or a TCP probe against a gRPC-only service would either fail to parse the response or miss real failures entirely — the protocol match isn't a style choice, it's the only way the check means anything.

Concept: An Env Var and a Running Dependency Are Two Different Requirements

frontend's `mustMapEnv()` panics at startup if `SHOPPING_ASSISTANT_SERVICE_ADDR` is unset — even though no `shoppingassistantservice` Deployment is ever created in this chart. The two are independent: the code only checks that the string exists at boot; it never dials that address unless the separate `ENABLE_ASSISTANT=true` flag is set and a user actually opens the assistant UI. So the fix is to keep a placeholder value in `frontendEnv` (matching Google's own manifest) and simply never deploy the workload or set the flag — satisfying a required variable is not the same as provisioning what it points at.

Build Sequence (the order that keeps you unblocked)

Helm chart first, on Google's existing images — then CI — then repoint the chart at your own registry only once a SHA exists. Switching the registry before CI is green means debugging two failures at once.

1. **Scaffold** — helm create charts/boutique, then delete every generated nginx sample file — keep only Chart.yaml, values.yaml, and _helpers.tpl.
2. **Redis (5a)** — Deployment + Service for redis-cart. First hyphenated key in values.yaml — `.Values.services.redis-cart` is invalid Go template syntax, so it's accessed with `{{ index .Values.services "redis-cart" }}` instead.
3. **cartservice (5b)** — Deployment + Service on port 7070, with `REDIS_ADDR=redis-cart:6379` wired from values — the exact DNS name the Service in step 2 created.
4. **frontend (5c)** — Deployment + Service with every `*_SERVICE_ADDR` from `frontendEnv` looped in via range, Service port 80 → container port 8080, HTTP probes on `/_healthz`.
5. **Render & review (5d)** — helm template boutique charts/boutique piped to a file and read like a code review — checking names, selectors, and every port pairing before anything touches the cluster.
6. **The rest, as a loop (5e)** — One templates/workloads.yaml ranges over the remaining 8 services in values.yaml, skipping frontend/cartservice/redis-cart (already hand-written). Inside a range, `.` is the loop item, so root chart values need `$` instead — `$.Values.image.registry`.
7. **Install & rollback (Lab 6)** — kubectl delete -f the Day 1 raw manifest first to avoid name collisions, then helm install. Bump a value, helm upgrade, then helm rollback boutique 1 to prove revision history works.
8. **CI pipeline (Lab 7)** — A test job gates an 11-way matrix build (fail-fast: false so one bad Dockerfile doesn't hide the other ten results), pushing SHA-tagged images to GHCR with the built-in GITHUB_TOKEN — no long-lived secret to leak or rotate.
9. **Point Helm at GHCR (Lab 8)** — Only after a SHA is confirmed in the registry: update image.registry and image.tag in values.yaml, helm upgrade, and confirm via kubectl describe pod that frontend is now pulling from ghcr.io, not Google's registry.

Issue 1 — Frontend Service port (80) vs. container port (8080) mismatch

Problem: The frontend container listens on 8080, but every other in-cluster caller (loadgenerator, and the browser via the Service) needs to reach it on port 80 — easy to silently get backwards when writing the Service template from scratch instead of copying it.

Diagnosis: Compared the Day 1 port-forward command (`'8080:8080'`, which targets the pod/container directly) against how loadgenerator's `FRONTEND_ADDR` was set in the original manifest (`'frontend:80'`, which targets the Service). The Service and the container are allowed to use different ports on purpose — that's what `'targetPort'` is for.

Solution: Set the frontend Service to `'port: 80'` with `'targetPort: 8080'`, and set loadgenerator's `FRONTEND_ADDR=frontend:80`. Verified with `'helm template'` before installing, rather than finding out via a failed request.

What I learned: A Kubernetes Service port and a container port are two independent numbers connected only by targetPort — conflating them is one of the most common Helm-chart authoring mistakes, and it fails silently (DNS still resolves, the connection just goes nowhere useful).

Issue 2 — emailservice Service port (5000) vs. container port (8080) mismatch

Problem: Same class of bug as Issue 1, but in the opposite direction and easy to miss because it only breaks one specific flow: checkoutservice calls `emailservice:5000`, but the container itself listens on 8080.

Diagnosis: Traced `EMAIL\_SERVICE\_ADDR` in checkoutservice's environment against emailservice's actual listen port in its source, rather than assuming symmetry with other services.

Solution: Set emailservice's Service to `port: 5000` with `targetPort: 8080`, keeping checkoutservice's env pointed at `emailservice:5000` unchanged.

What I learned: Port mismatches between services don't fail at `helm install` time — they fail at request time, on exactly one code path (PlaceOrder's fake email step), which makes them easy to miss during a quick smoke test.

Issue 3 — Frontend pod panics for a service that isn't even deployed

Problem: frontend's `mustMapEnv()` panics if `SHOPPING\_ASSISTANT\_SERVICE\_ADDR` is unset — even though no `shoppingassistantservice` Deployment exists in this chart and the feature is gated off (`ENABLE\_ASSISTANT=true` is never set).

Diagnosis: Recognized this as the same fail-fast pattern hit on Day 1 with `PRODUCT\_CATALOG\_SERVICE\_ADDR` — the code checks that the variable exists, not that the service behind it is reachable or even deployed.

Solution: Kept `SHOPPING\_ASSISTANT\_SERVICE\_ADDR` in `frontendEnv` in values.yaml pointing at a placeholder address, without deploying a matching workload — matching what Day 1's manifest already did.

What I learned: A required env var and a required running dependency are two different things — you can satisfy the first without provisioning the second, as long as the code path that would use it stays disabled.

Issue 4 — GHCR image pull fails after switching Helm off Google's registry

Problem: After pointing `values.yaml` at `ghcr.io/<owner>/<repo>` and running `helm upgrade`, pods went to `ImagePullBackOff` instead of picking up the newly built images.

Diagnosis: Ran `kubectl describe pod` on the affected pod and read the Events section, then checked three things in order: whether the SHA tag actually existed in GHCR, whether the owner/repo string was lowercase (GHCR rejects uppercase), and whether the package visibility was still Private.

Solution: In this environment the fix was one of: correcting a tag typo, lowercasing the image path, or switching the GHCR package visibility to Public so the cluster could pull without an imagePullSecret.

What I learned: Same debugging discipline as CrashLoopBackOff — describe pod's Events section names the actual cause; ImagePullBackOff almost always traces back to the tag, the casing, or the registry's visibility setting, not a Kubernetes problem.

Issue 5 — Helm resolved charts/boutique as a remote repo instead of a local folder

Problem: Running `helm upgrade --install boutique charts/boutique` (no leading `./`) failed with `Error: repo charts not found` instead of installing the local chart.

Diagnosis: Recognized that Helm treats a bare `charts/boutique` as `/chart-name` — the same syntax used for a chart registered via `helm repo add` — rather than a filesystem path, unless the path is unambiguous.

Solution: Re-ran the command with an explicit relative path: `helm upgrade --install boutique ./charts/boutique`. Also caught a related typo later (`charts/boutique`, dot attached directly to the folder name) which Windows/Git Bash read as a hidden directory that doesn't exist.

What I learned: Helm's argument parsing is ambiguous between local paths and repo references — a leading `./` removes the ambiguity entirely and costs nothing to always include.

Issue 6 — nil pointer evaluating interface {}.enabled on an optional template

Problem: Added an optional `templates/httproute.yaml` (Gateway API HTTPRoute) guarded by `{{- if .Values.httpRoute.enabled -}}`, and `helm lint` failed with `nil pointer evaluating interface {}.enabled` because `httpRoute` didn't exist anywhere in `values.yaml` yet.

Diagnosis: Understood that Helm's template engine can't call `.enabled` on a key that doesn't exist at all — `.Values.httpRoute` was nil, not an empty map, so the second field access failed before the `if` could even short-circuit.

Solution: Rewrote the guard as `{{- if and .Values.httpRoute .Values.httpRoute.enabled -}}` — `and` short-circuits on the first falsy value, so a missing `httpRoute` block now safely skips the whole resource instead of crashing the render.

What I learned: Any optional Kubernetes resource in a chart needs this two-step guard pattern — checking that the parent key exists before checking a field on it — or the chart breaks for anyone who doesn't happen to define every optional block.

Issue 7 — 'exists and cannot be imported' — Helm refused to adopt resources kubectrl already created

Problem: `helm install boutique charts/boutique` failed with `Service "cartservice" ... cannot be imported into the current release: invalid ownership metadata; missing key "app.kubernetes.io/managed-by"` — because Day 1's `kubectrl apply` had already created a Service of the same name outside of Helm.

Diagnosis: Recognized this as a safety feature, not a bug: Helm refuses to silently take ownership of a live resource it didn't create, since doing so could overwrite something another process depends on.

Solution: Deleted the leftover Day 1 resources first (`kubectrl delete -f release/kubernetes-manifests.yaml`) rather than manually patching ownership labels onto them — cleaner for a chart that's meant to fully own this namespace.

What I learned: Helm's ownership check is exactly the kind of guardrail that prevents a GitOps-style tool from clobbering manually-managed infrastructure — worth explaining as a feature, not working around it without understanding why it exists.

Issue 8 — Hyphenated key access failed in the Redis template

Problem: Writing `.Values.services.redis-cart.replicas` directly in a template failed to render — Go's template language parses the bare hyphen as a subtraction operator (`redis` minus `cart`), not part of an identifier.

Diagnosis: Traced the failure to Helm/Go template syntax rules rather than a `values.yaml` problem — the key itself (`redis-cart`) was fine; only dot-notation access to it was invalid.

Solution: Used the map index function instead: `{{- $redis := index .Values.services "redis-cart" -}}` at the top of the file, then referenced `$redis.replicas`, `$redis.image`, etc. throughout — sidesteps the parser issue entirely.

What I learned: Any `values.yaml` key with a hyphen, and Kubernetes/Helm convention uses plenty (`redis-cart`, `node-pool`, etc.), needs index-based access — this is a routine gotcha, not a sign the chart's structure is wrong.

Issue 9 — InvalidImageName across almost every pod after switching to GHCR

Problem: After updating `image.registry` in `values.yaml` and running `helm upgrade`, nearly every new ReplicaSet came up as `InvalidImageName` while the old pods stayed running — doubling the pod count and leaving the release in a half-migrated state.

Diagnosis: Ran `kubectrl describe pod` on a failing pod and read the rejected image string directly off the `Image:` field, rather than guessing from the values file. The registry path was `ghcr.io/<owner>/Online-Shopping-Store-Devops` — mixed-case.

Solution: Lowercased the entire registry path in `values.yaml` (`ghcr.io/<owner>/online-shopping-store-devops`) and re-ran `helm upgrade`. Old pods were left running throughout, since Kubernetes never tears down a working ReplicaSet in favor of one that can't even be scheduled.

What I learned: Docker/OCI registry paths are case-sensitive and lowercase-only by spec — this rule applies to the full path (owner and repo, not just the image name), and GHCR rejects the whole string rather than normalizing it silently.

Issue 10 — CI push failed with 'ERROR: failed to build: unknown blob'

Problem: The matrix build for one service completed and started pushing layers to GHCR, then failed partway through with 'unknown blob' — a generic error from the registry rejecting a layer reference during a concurrent multi-service push.

Diagnosis: Recognized this as a BuildKit cache/registry desync rather than a Dockerfile problem, since the same build had already succeeded locally in earlier layers of the same log — the failure was specific to the push step under parallel matrix load.

Solution: Added an explicit 'docker/setup-buildx-action@v3' step between the GHCR login and the build-push step, giving each matrix job its own isolated builder instance instead of relying on the runner's implicit default one.

What I learned: Parallel matrix jobs sharing an implicit builder can race on the registry's blob store; pinning a dedicated Buildx instance per job is the standard fix, and it's worth being able to say why the error looked like a registry problem but was actually a CI concurrency problem.

Issue 11 — Helm template broke on . vs \$ scoping inside a per-service file

Problem: '{{- toYaml .Values.defaults.resources | nindent 12 }}' inside emailservice's template failed to resolve correctly, even though the same 'defaults.resources' block rendered fine elsewhere in the chart.

Diagnosis: Traced it to Go template scoping: once a file defines a local variable like '{{- \$svc := .Values.services.emailservice }}' and uses '.' inside nested blocks (like 'range'), the meaning of a bare '.' can shift away from the chart root — so '.Values' no longer reliably points where it looks like it does.

Solution: Replaced the bare '.' with the root-context anchor: '{{- toYaml \$.Values.defaults.resources | nindent 12 }}'. '\$' always refers to the top-level chart context regardless of how deep the current block is nested, unlike '.'.

What I learned: In any Helm template with nested 'range' or 'with' blocks, '.' is unreliable for reaching root-level values — '\$.Values...' is the safe habit to default to, not just a fix for one broken file.

Issue 12 — recommendationservice stuck in CrashLoopBackOff after the image itself was fixed

Problem: Once Issue 9 was resolved, recommendationservice still wouldn't stabilize: 'kubectl describe pod' showed 'Exit Code: 137' with repeated 'Liveness probe failed: ... context deadline exceeded', and two pods (one old, one new) were visible at once, which looked like a stuck deployment.

Diagnosis: First confirmed the two-pod state was an ordinary rolling update — Kubernetes never removes the last working pod until a replacement passes its readiness check, so an old and a new pod coexisting briefly is expected, not a bug. The real issue was in the surviving new pod's Events: liveness probes were timing out because the Python service takes longer than the 1-second default probe timeout to finish starting.

Solution: Added 'initialDelaySeconds: 15' and 'timeoutSeconds: 5' to both the readiness and liveness probes in recommendationservice's template, giving the slow-starting Python process room to boot before Kubernetes started judging it.

What I learned: Exit code 137 during startup doesn't always mean OOM — a liveness probe with no grace period can kill a genuinely healthy but slow-booting process just as easily, and the Events section is what tells the two apart. A transient 'two pods' sighting during a rollout is a status update, not an incident.

Day 3 — Terraform & AKS

Day 3 replaces kind with real Azure infrastructure: a Terraform-provisioned AKS cluster, driven by the same Helm chart from Day 2. It's the only day with a direct cost, and the only day that ends with a mandatory teardown step.

Term	Meaning	Today
------	---------	-------

Provider	Plugin that talks to a cloud API	azurerem
Resource	One object declared in code	azurerem_kubernetes_cluster, etc.
State	Terraform's record of what actually exists	terraform.tfstate — never committed to Git
Plan	A diff: what would change if applied	terraform plan -out=tfplan

Build Sequence

Budget alert first, always — before any resource that could accrue cost exists. Terraform infrastructure second. The same Helm chart from Day 2, unmodified, pointed at the new cluster last — proof the packaging was portable, not rebuilt.

10. **Budget alert** — A \$20 spend alert on the subscription, created before the first `terraform apply` that provisions AKS — not after.
11. **Throwaway state lab** — A scratch resource group applied, planned, diffed, destroyed, and recreated outside the real infra/ folder — purely to read terraform.tfstate and feel what state actually tracks before it matters.
12. **Write infra/** — Four files — versions.tf, variables.tf, main.tf, outputs.tf — declaring a resource group, VNet, subnet, user-assigned identity, AKS cluster, and ACR, with AcrPull granted to the kubelet identity specifically.
13. **Plan, read, apply** — terraform plan -out=tfplan reviewed line by line before terraform apply tfplan — confirming node size, count, and that no resource is being replaced instead of updated.
14. **Point kubectl at Azure** — az aks get-credentials switches the active context from kind to the new AKS cluster; kubectl get nodes confirms two real nodes, Ready.
15. **Same chart, real LoadBalancer** — helm upgrade --install boutique charts/boutique --set services.frontend.type=LoadBalancer — no template changes, just overriding one value to get a real public IP instead of kind's permanent <pending>.
16. **Verify, then destroy** — Checkout confirmed from a phone over the public IP and screenshotted, then terraform destroy every session before stopping the Codespace — a forgotten cluster, not the lab itself, is what gets expensive.

Issue 1 — AKS rejected the planned node size — B-series VMs blocked for system pools

Problem: `terraform apply` failed with a 400 listing dozens of allowed SKUs, rejecting the burstable B-series size originally planned for the node pool.

Diagnosis: Read the error's own explanation rather than guessing: AKS enforces a 2 vCPU / 4 GB minimum on system node pools, and explicitly excludes B-series burstable VMs from that pool regardless of their nominal size — because core components like CoreDNS need guaranteed, non-throttled CPU, not a burst credit that can run out.

Solution: Switched `node\_size` to a standard general-purpose D-series SKU instead of a B-series one, keeping node_count at what the budget allowed.

What I learned: A VM family being 'big enough' on paper doesn't mean it's eligible for a given role — AKS system pools specifically require guaranteed-performance CPU, and that's a category restriction, not a sizing one.

Issue 2 — terraform apply timed out contacting Azure: 'context deadline exceeded'

Problem: The very first `terraform apply` failed while Terraform was still listing the subscription's available resource providers, before creating anything — `populating Resource Provider cache: ... context deadline exceeded`.

Diagnosis: Recognized this as a network-layer timeout between the local machine and Azure's management API during provider initialization, not a problem with the Terraform code itself — nothing in the plan had been touched yet.

Solution: Added ``resource_provider_registrations = "none"`` to the ``provider "azurerm"`` block, skipping the full provider-listing step Terraform normally does on every run, and cleared the local ``.terraform/`` cache before re-initializing.

What I learned: Terraform's provider initialization does more network work up front than the plan itself needs — on a slow or congested connection, that preliminary step can fail before the interesting part even starts, and it has its own dedicated escape hatch.

Issue 3 — `kubelet_identity[0]` broke under the current `azurerm` provider version

Problem: The ``azurerm_role_assignment`` granting `AcrPull` to the cluster's kubelet identity referenced ``azurerm_kubernetes_cluster.this.kubelet_identity[0].object_id`` — a pattern copied from older documentation — and failed to evaluate.

Diagnosis: Checked the resource schema for the pinned ``azurerm`` provider version rather than assuming the old syntax was still correct: in the provider version this chart used, ``kubelet_identity`` had changed from a single-item list to a direct object block, so indexing into it with ``[0]`` no longer matched the schema.

Solution: Removed the list index and referenced the attribute directly:
``azurerm_kubernetes_cluster.this.kubelet_identity.object_id``.

What I learned: Terraform provider schemas change between major versions in ways that don't always throw an obviously-versioned error — when a copied snippet fails on an attribute reference, checking the current provider's schema is faster than assuming the code is wrong for some other reason.

Issue 4 — CNI Overlay mode rejected on a case-sensitivity mismatch, needed to avoid subnet IP exhaustion

Problem: Setting ``network_plugin_mode = "Overlay"`` failed validation (``expected ... to be one of ["overlay"]``) — and the setting existed in the first place because the planned subnet (a /24, ~251 usable IPs) would have been exhausted quickly under legacy Azure CNI, which assigns every pod its own routable IP directly from the subnet.

Diagnosis: Understood the underlying design reason before just fixing the typo: AKS's legacy CNI mode consumes one subnet IP per pod, so a small subnet caps the cluster's real pod capacity far below what node count alone would suggest; Overlay mode assigns pods IPs from a separate CIDR instead, decoupling pod scale from subnet size.

Solution: Corrected the value to lowercase `""overlay""` (Terraform providers frequently expect exact-case enum strings, not case-insensitive matches) and added a dedicated ``pod_cidr`` for the overlay network.

What I learned: A validation error on an enum-like field is often just casing — but it's worth understanding why the field was being set at all; here the real story was subnet sizing, not the typo, and that's the part worth being able to explain.

Issue 5 — AKS creation failed on regional vCPU quota — root cause was the subscription tier, not the SKU

Problem: After fixing Issues 1–4, ``terraform apply`` still failed creating the cluster: ``ErrCode_InsufficientVCPUQuota ... requested 2, remaining 0 for family standardDDSV5Family``. Switching to a different D-series generation hit the same class of error again.

Diagnosis: Stepped back from SKU-by-SKU trial and error to check the actual constraint: an Azure for Students subscription carries a hard regional cap (a handful of total vCPUs across the whole account), blocks self-service quota increase requests entirely, and excludes several newer VM families outright — so no amount of retrying different SKUs in the same family class would fix it.

Solution: Switched to an older, broadly-available generation (``Standard_D2s_v3``) that fits inside the student tier's total vCPU cap, and kept ``node_count`` at exactly 1 rather than 2 so the cluster's total request stayed under the hard ceiling.

What I learned: A quota error names the specific SKU family, but the real constraint can be the subscription tier itself — recognizing 'this account can't request more, no matter which SKU I pick' turns a loop of failed applies into one deliberate, informed choice.

Day 4 — GitOps & Observability (fill in as completed)

Suggested entries to complete once Day 4 is done: what happened when a Deployment was deleted by hand with ArgoCD auto-sync enabled, versus the Day 1 pod-delete behaviour; a PromQL query that didn't return what was expected and why; the alert rule written and how it was deliberately triggered.

5. Key Skills Demonstrated

- Container internals: multi-stage Docker builds, distroless runtime images, CGO/static binaries.
- Kubernetes core objects: Pods, Deployments, ReplicaSets, Services, DNS-based service discovery.
- Debugging methodology: get → describe → logs → exec, used systematically rather than guessed.
- Helm chart authoring: templating, values files, helpers, release rollback.
- CI/CD: matrix builds, container registries, immutable image tagging strategy.
- Infrastructure as Code: Terraform providers, resources, state, plan/apply/destroy discipline.
- Cloud networking basics: LoadBalancer vs ClusterIP, public IP exposure, managed identities.
- GitOps principles: declarative sync, drift correction, and how it differs from imperative `kubectl apply`.
- Observability: PromQL fundamentals, dashboard design, alerting on real signals.
- Cost discipline: budget alerts, resource sizing, and tearing down cloud infrastructure when not in use.

6. Interview Q&A Preparation

These are the questions this project is designed to make answerable from memory, without notes.

- Walk me through what happens when a user clicks 'Add to Cart'.
- `cartservice` dies at 2am. How do you find out, and what do you do?
- Why did you tag images with the git SHA instead of `:latest`?
- What is Terraform state, and why must it never live in Git?
- What does ArgoCD give you that a `kubectl apply` step in CI does not?
- Your pod is stuck in Pending. Walk me through diagnosing it.
- Why is the frontend a LoadBalancer while the backends are ClusterIP?
- What would you change before putting this in front of real customers?

7. Honest Scope & Next Steps

This project was built end-to-end in four focused days to learn the full delivery pipeline hands-on — it is not four days of Kubernetes mastery, and it isn't presented as such. In an interview the accurate framing is: "I built this in four days to understand the whole pipeline end to end, and here is what I would change with more time." That is a credible claim, and it opens a better conversation than overstating expertise.

What I would improve with more time

- Add TLS/HTTPS termination in front of the frontend Service.
- Move Redis to a PersistentVolumeClaim or a managed cache for durability.
- Add NetworkPolicies to restrict pod-to-pod traffic by default.
- Move Terraform state to a remote backend (e.g. Azure Storage) with locking.
- Add a HorizontalPodAutoscaler for frontend and `cartservice`.
- Expand test coverage in the CI pipeline beyond build/lint.